VoIP App — Feature Spec v0.3

Every feature below has a **plain-language description**. These aren't just notes — they're the actual text that should ship in the app as tooltips or inline help. Rule: if a user has to google what a setting does, the setting is broken.

---

0. Core Principles

Principle	Meaning
User choice	Every automatic behavior has a manual override. Auto is the default; power users get the switches.
Trust	Nothing happens to a user's data, connection, or reputation that they weren't told about in plain words first.
Encrypted end to end, always	Decryption happens on the user's device. No mode, server, or fallback path ever changes this.
Explain everything	No feature ships without a one-sentence description in the UI.
Your data is yours	No central server required. Data lives where the user chooses.
Lightweight	Native code, no browser engine, low idle CPU and RAM.

---

1. Audio Suite

A full processing chain on your microphone, in order. Each stage toggles independently, and the whole chain saves as a preset.

Cleanup stage

Noise Gate

> Mutes your mic when you're not talking. Stops your keyboard, fan, and background chatter from being heard between sentences. - Controls: threshold, attack, hold, release - Live meter showing what's passing through

Noise Suppression

> Removes steady background sound *while* you're talking — fans, air conditioning, computer hum. Different from the gate, which only works between words. - ML-based (RNNoise-class), strength slider - Warning at high strength: can make voices sound thin

High-Pass Filter

> Cuts low rumble — desk bumps, footsteps, breath thumps on the mic. - Adjustable cutoff, default ~80 Hz

Low-Pass Filter

> Cuts high hiss and sharpness. Useful for cheap mics with a lot of static.

Echo Cancellation

> Stops the other people's voices from being picked up by your microphone and sent back to them as an echo. Essential if you use speakers instead of headphones. - On by default, with a manual off switch for users on headphones who want the lowest possible processing - Adaptive filter length; must handle device hot-swap without needing to re-converge from scratch

De-esser

> Softens harsh "S" and "T" sounds without dulling the rest of your voice.

Tone stage

Parametric EQ

> Shapes how your voice sounds — add clarity, remove boominess, cut a specific annoying frequency. - Multi-band, draggable curve, per-band type (bell/shelf/notch) - Starter presets: Warm, Clear, Broadcast, Thin Mic Rescue

Compressor

> Evens out your volume. Quiet talking gets louder, shouting gets quieter, so you never blast the people listening. - Threshold, ratio, attack, release, makeup gain - Gain reduction meter

Limiter

> A hard ceiling on your volume. Catches sudden loud noises before they hit anyone's ears.

Gain / Auto Gain

> Sets your overall mic level. Auto keeps you at a consistent volume without you touching anything.

Creative stage

Pitch Shift

> Raises or lowers your voice pitch. - Formant-preserving option (natural instead of chipmunk/demon)

Reverb

> Adds a sense of space, like talking in a room or hall. Mostly for fun. - Room size, damping, wet/dry mix

Voice Presets / Chains

> Save your whole setup as a named preset. Export it, share it, load someone else's.

Utility

Push-to-Talk / Voice Activation

> Choose how your mic opens. Push-to-talk sends audio only while you hold a key. Voice activation opens automatically when you speak.

- **Push-to-talk default key: Caps Lock**, held — not a toggle
- Config menu allows *any* input to be rebound, and allows PTT or mute/deafen to be switched to toggle behavior per user
- Release delay slider, so the tail of your last word isn't cut off
- **Voice activation**: sensitivity slider with a live meter showing where the threshold sits against your actual voice
- Mute and deafen, each independently bindable
- Global hotkeys work while the app is unfocused

Implementation note — Caps Lock is a stateful toggle key at the OS level, so capturing it as a momentary key needs care: a low-level hook on Windows, HID access on macOS, and compositor/portal support on Wayland. Some users have also remapped it to Ctrl or Escape. Detect capture failure and prompt for a different key instead of silently not working.

Live Monitoring

> Hear your own mic in your headphones, with a readout of how much delay you're adding.

Mic Test & Record Preview

> Record five seconds and play it back so you hear exactly what everyone else hears.

Device Hot-Swap

> Change microphone or headphones mid-call without dropping.

Plugin Host *(consider — big Linux differentiator)*

> Load your own LV2 / LADSPA / VST audio plugins into the chain.

Receive side (per person)

Per-User Volume & EQ

> Turn individual people up or down, or fix someone's harsh mic on your end without asking them to change anything.

Ducking

> Automatically lowers everyone else slightly when one person is talking, so overlapping voices stay clear.

---

2. Lost Audio Recovery

Four layers. Most losses never reach the user; the visible feature is the last resort.

Layer	What it does
Forward error correction	Sends a low-quality backup copy of each moment of audio alongside the real one. Small gaps get filled instantly with no delay.
Retransmit request	If a chunk goes missing, your app asks the sender to resend it before it's needed.
Adaptive jitter buffer	Holds a fraction of a second of audio in reserve to smooth out unstable connections, and shrinks again when things are stable.
Replay	If audio was still lost, it can be recovered and played back.

Replay — user-facing

> When someone's audio drops out, the missing piece can be recovered and played back to you. Choose whether it plays automatically or waits for you to tap a button.

- Modes: **Auto-play** / **Ask me** (popup button) / **Off**

- Replay history panel: last N recovered clips, replayable manually
- **Catch-up playback**: recovered audio plays slightly sped up so you rejoin the live conversation instead of falling behind
- **Buffer window**: how much audio is held locally, user-configurable (e.g. 10–60 seconds)
- **Ephemeral by design**: the buffer is encrypted, in memory, and discarded on a rolling basis. State this loudly in the UI — otherwise "the app records you" becomes the story.

---

3. Connection & Trust Model

Who you connect to directly

Known contacts → Direct P2P

> A direct connection to someone on your contact list. Fastest and most private, but you and they can see each other's IP address — the same as any direct connection on the internet. Best for people you already know.

Everyone else → Relay (default)

> Your connection goes through a relay server that passes along encrypted data. Nobody learns your IP address, and the relay can't read or hear anything. Slightly slower, and the safe default for strangers.

Anonymity mode

> Routes your connection through several relays, each one only knowing the step before and after it. The strongest privacy available, at the cost of noticeable delay.

Mode is chosen automatically based on whether someone is a known contact, and always manually overridable per conversation. The active mode is visible in the call UI with a one-line explanation.

Group calls

- **Up to ~5 people** — direct mesh, no infrastructure
- **Beyond that** — SFU relay. Each person uploads once; the relay forwards encrypted frames to everyone. The relay holds no keys and cannot hear anything.
- Community owners can self-host their own SFU. Project-hosted relays exist as fallback.

Encryption

- **1:1** — Double Ratchet style, forward secrecy
- **Groups** — MLS (RFC 9420), scales cleanly to 100+ members

- **Media through a relay** — SFrame; the relay routes encrypted frames without keys
- **Multi-device** — new devices added by approving them from an existing device
- **Backup & recovery** — encrypted backup blob, restorable with a recovery phrase or split among trusted contacts

What a relay operator can see — and the commitment

Encryption hides *what* you say, never *that* you connected. A relay necessarily handles addresses to route packets at all. Design requirements:

- Blinded session tokens, no persistent user identifiers at the relay
- No connection logs retained
- The relay never holds a key to anything
- All of the above stated publicly and plainly, not buried in a privacy policy

---

4. Finding People (Discovery & Identity)

Two separate problems: **reaching** someone whose IP keeps changing, and **knowing the key is really theirs**.

Reaching people

QR code / invite link — primary method

> Show your code, they scan it, you're connected. Nothing about you is published anywhere. - Surfaced directly on your own profile screen — one tap from anywhere, not buried in settings - Position is user-configurable via the UI editor, including hiding it entirely

DHT (distributed lookup) — reconnection

> A shared, decentralized address book spread across everyone's devices. It lets your contacts find you again when your internet address changes, without any company holding the list. - Records are **encrypted and signed** — the network stores them without being able to read who they belong to - Blinded lookups so nodes can't tell who you're searching for - Protections against fake-node flooding (sybil/eclipse resistance) are a hard requirement before this ships

Opt-in directories — public communities only

> Communities that want to be discovered can list themselves. Individual users are never listed without explicitly choosing it. - No global username search by default. That's a permanent privacy hole, and it's how harassment scales.

Knowing it's really them

Verification code

> A short code both of you can compare in person, by voice, or over another app. If it matches, nobody is intercepting you.

Contact introduction

> A mutual friend can vouch for someone by passing their key through your existing encrypted chat, so you don't have to verify from scratch.

Trust-on-first-use with a persistent warning if a contact's key ever changes.

Infrastructure position

Project-hosted infrastructure is **fallback, not foundation**: bootstrap nodes, fallback relays, optional directories. If it all disappears, existing contacts keep working. Only *new* first contact gets harder. That's a fundamentally different failure mode than a centralized service going down.

---

5. File Transfer

Default: peer-to-peer, no size limit.

> When you share a file, it isn't uploaded anywhere. It stays on your device, and the app sends everyone a link to it. The file only transfers when someone actually asks to download it, straight from your machine to theirs.

How it works

- Files are **content-addressed** — identified by a hash of their contents rather than a location. Two people who already have the same file never re-download it.
- Files are **chunked**, each chunk hashed and verified. Transfers resume after disconnects instead of restarting.
- Each file is encrypted with its own key, delivered inside the encrypted message. Any peer or server can hold a copy without being able to read it.
- Because chunks verify independently, a download can pull from **several peers at once**. In a large community a popular file gets *faster* the more people have it — the opposite of a central server.

Availability

If the sender goes offline, their file goes with them. All solutions are opt-in:

Seeding

> After you download a file, you can keep helping share it with others. Off by default; nothing is stored or shared without you turning it on.

Seeding scope — required setting

> Choose who you're willing to help share files with: this conversation only, this community only, your contacts, or anyone. - Default: **this community only** - Scope is enforced at the protocol level, not just hidden in the UI

Anonymous seeding

> Share files through relays so the people downloading never see your IP address. Slower, and uses more bandwidth, but nobody can tell who's hosting what.

Pinning

> Ask a device or server you trust to hold a copy of a file so it stays available even when the person who shared it is offline.

Availability status — always visible on every file: `Available` · `Sender offline — will resume when they're back` · `No longer shared`

Server-hosted mode (opt-in)

- Per-community toggle, off by default
- Max file size, total storage quota, per-user quota
- Retention: keep forever / expire after N days / expire when unopened
- Stored **encrypted** — the server holds bytes it cannot read
- Admin view of usage, manual purge

Details that will bite if skipped

- **Mobile file access** — Android scoped storage needs persisted URI permissions; iOS needs security-scoped bookmarks. Storing a raw file path breaks on app restart.
- **Moved or deleted files** — re-verify by hash before offering a download; mark unavailable rather than failing mid-transfer.
- **Background transfers** — Android needs a foreground service; iOS needs resume-on-open.

- **Thumbnails and previews** — generated by the sender, sent inline as small data, so chat isn't empty before anyone downloads.
- **Bandwidth controls** — upload cap, per-transfer throttle, queue, pause/resume, "only on Wi-Fi" for mobile.

---

6. Network Privacy Tools

Bring your own VPN

> Route the app's traffic through your own VPN by importing a WireGuard configuration file from a provider you already pay for. Only this app's traffic is affected — the rest of your device is untouched, and it won't fight with a VPN you already have running.

- Userspace WireGuard inside the app, not a system-level VPN. Avoids the one-tunnel-at-a-time conflict on Android and iOS, and keeps the project out of the business of operating exit nodes.
- Import, store, and switch between multiple configs. Per-mode routing: send file transfers through the VPN while leaving voice direct, or the reverse.
- **Key storage** — config files contain private keys. Stored in the OS keystore/keychain, never in plaintext, never included in theme exports or backup blobs.
- **Kill switch** — if the tunnel drops, app traffic stops rather than falling back to your real connection.

Port forwarding — what it does and doesn't affect

Every VPN works with this app. Port forwarding only changes what you can host, never how protected you are.

Some providers allow incoming connections (port forwarding); many don't. Either way:

	Without port forwarding	With port forwarding
Traffic encrypted	Yes	Yes
Your IP hidden	Yes	Yes
Join calls, send messages	Yes	Yes
Download files	Yes	Yes
Others connect directly to you	No — routed through a relay	Yes
Seeding files	No	Yes

In-app wording

> Your VPN provider doesn't allow incoming connections. Everything still works — your traffic is fully encrypted and your IP address is still hidden. The only difference is that other people can't connect straight to you, so you'll be routed through a relay and you won't be able to help share files.

This is a capability limit, not a privacy weakness. The UI must say so in exactly those terms, or users will assume something is broken or unsafe.

Automatic port negotiation

> If your VPN supports port forwarding, the app requests and renews the port for you in the background, and updates itself when the port changes. Nothing to configure.

- Built-in NAT-PMP client, auto-renewing (some providers expire mappings in under a minute)
- Handles port changes on reconnect without user action

Reachability status — always visible, never discovered through failure: `Direct` · `Relayed — VPN doesn't allow incoming connections` · `Relayed — network blocks direct connections`

---

7. Interface & Customization

This is a **full UI editor**, not a theme picker. Treat it as a headline feature, not a settings page.

- **Layout editor** — move, resize, add, remove, and reorder every panel. Save named arrangements.
- **Profile layout** — the user's own profile screen is editable like everything else. QR code, status, presence, custom fields: reposition or hide any of them.
- **Scale** — independent of OS scaling
- **Fonts** — family and size, including a bundled dyslexia-friendly option
- **Color themes** — full palette control, light/dark/custom
- **Export & import** — one file carries layout + colors + fonts + profile arrangement
- *Security requirement:* theme files are declarative data only. No scripting in a theme, ever.
- **Presets for people who don't want to build one** — a good default plus a gallery, so customization is an invitation and never a chore
- **Statuses & presence** — custom text, per-community identity
- **Accessibility** — screen reader support, on-device live captions, keyboard-only navigation, reduced motion

---

8. Message Delivery

Messages send when both people are reachable. No copy sits on anyone's server waiting by default.

Outbox

> Messages you send while someone is offline stay on your device and send themselves the moment they come back online. You don't have to do anything.

Multi-device catch-up

> If you have a desktop that's usually on, it receives your messages while your phone is asleep, then hands them to your phone when it reconnects.

Delivery Routes — the tier system

A single ordered list. Each route can be turned on or off, reordered by dragging, and pointed at specific nodes you choose. The app walks the list top to bottom and uses the first one that works.

Default order:

#	Route	Description shown in-app
1	**Direct**	Straight to their device, if they're online right now. Nothing is stored anywhere.
2	**My other devices**	Your desktop or tablet holds messages while your phone is asleep, then hands them over when it wakes up.
3	**A node you trust**	A machine you or a friend runs — a home server, a spare PC, a Raspberry Pi. It holds sealed messages it can't read.
4	**Community node**	The community you're talking in holds it for you. Same sealed storage, run by the people you're already talking to.
5	**Project relay**	Our fallback. Used only when nothing above is available, and it holds no more information than any other node.

Per-route controls - On / off - Node picker: add by address, QR, or auto-detect on your local network - Live status per node: reachable, response time, last used, storage used, storage cap - Remove or replace at any time — messages waiting there are re-routed, not lost

Redundancy setting

> Choose whether a message goes to the first available route, or to several at once for reliability. - **First available** — least metadata exposure - **Fan out** — the same sealed blob is dropped at multiple nodes, first delivery wins, the rest expire. No single operator ever sees all your traffic, at the cost of more nodes knowing *something* arrived.

Strict mode

> Turn off every route except Direct. Messages only ever move when both of you are online at the same moment, and nothing is stored anywhere, ever.

Per-contact overrides

> Use different routes for different people. Direct only for close contacts, relayed for everyone else.

Server-side opt-in

> Any self-hosted server or community node chooses whether it acts as a relay or mailbox for others. Off by default. Operators set storage caps, per-label rate limits, expiry, and whether they serve only their own community or anyone.

- Writes accepted only to labels currently registered with that node — unknown labels are rejected without being stored, so a public node can't be filled by strangers
- Operator dashboard: storage used, labels held, bandwidth, with no ability to see contents or identities

The part that's easy to get backwards

The recipient chooses where their mail is deposited, not the sender.

Your route list is really your *inbox* configuration, published to your contacts as part of your profile. When someone messages you, their app reads your published list and drops the sealed envelope at the highest-priority spot you've made available. Your privacy settings are enforced by your own list — nobody can decide to leave your mail somewhere you didn't authorize.

The sender controls only their **outbox**: how long to keep retrying, and whether to give up. Both halves are settings; they just live on different sides.

Delivery status — plain wording, always visible: `Sending` · `Waiting for them to come online` · `Delivered`

---

9. Chat & Communities

Starting a call

Silent call

> Starts a call without ringing anyone. It shows up quietly in the chat, and whoever's around can join when they notice. Good for group chats where you don't want to buzz twenty phones.

- **Default: Shift + click the call button**

- Config menu: expose it as a separate always-visible button, or make silent the *default* behavior with a normal ring on Shift instead
- The call appears as a persistent bar in the channel — "Voice active · 3 people" with a join button — rather than a notification
- **Escalate**: if nobody joins, the caller can promote a silent call to a ringing one without restarting it
- Works in 1:1 too, as a knock rather than a ring: *"I'm around if you want to talk"*

Receiver settings always win

> You decide which chats are allowed to ring you, no matter how the call was started.

- Per-chat and per-community ring settings: always ring / silent only / never
- A caller can never force a ring through someone's do-not-disturb. Silent is a courtesy on the sender's side; the guarantee lives on the receiver's side.
- **Group chats** up to ~100, no server required
- **Communities** with channels, roles, permissions
- **Threads and replies**
- **Search** over your own history, indexed locally on your device
- **Moderation tools** — block, mute, report, per-community ban lists (needed for app store review as much as for users)
- **Bots / API** — local-first, running on a user's machine rather than a company's

---

10. Platform Strategy

One codebase, thin platform layers.

Layer	Shared?
Audio pipeline, encryption, networking, protocol, state	Shared (Rust core) — roughly 80–85% of the code
Desktop UI (Linux, Windows, macOS)	Shared (Slint — declarative markup doubles as the theme format)
Mobile UI (Android, iOS)	Native (Compose / SwiftUI) calling the same core over FFI
Audio backends	Per-platform: PipeWire/ALSA, AAudio, WASAPI, CoreAudio
Push notifications	Per-platform: FCM, APNs/PushKit, persistent socket on desktop
Packaging	Per-platform: Flatpak/AppImage/deb, APK, MSI, dmg

Ship order: Linux + Android first — shared toolchain, primary targets, and Android takes the Rust core directly through the NDK. Windows next. macOS/iOS last; they need separate signing, store review, and CallKit/PushKit work regardless.

One release pipeline builds all targets from the same commit. Features land everywhere at once; only packaging differs.

---

11. Deferred (v1.5+)

Sequenced, not cut:

- Video calls
- Screen share with audio
- SFU relay for large calls
- Communities at full scale
- Plugin host
- Bots / API
- Anonymity mode (multi-hop)

---

12. Open Questions

1. **Identity model** — usernames, keypairs only, or both? What's the recovery story when someone loses every device?
2. **Relay funding** — fallback relays cost real bandwidth. Donations, community-run, or volunteer?
3. **Push on iOS** — Apple's servers are unavoidable for background calls. Confirm the encrypted-payload design before committing to iOS.
4. **Federation** — can two self-hosted communities talk to each other, or are they islands?
5. **Moderation without content access** — you can't scan what you can't read. What are the actual tools, and what's the answer when a store reviewer asks?

---

13. Proposed — Not Yet Confirmed

Suggested, awaiting your call. Nothing here is decided.

- **Notification controls** — per-community and per-channel granularity, schedules, do-not-disturb
- **Message lifecycle** — edit, delete, disappearing messages, pinned messages, saved / note-to-self

- **Link previews** — privacy trap: fetching a preview leaks your IP to that website. Should be sender-generated or off by default.
- **Reactions, custom emoji, soundboard** — the soundboard is nearly free given the audio chain already exists
- **Voice messages** — short async audio clips
- **Signed & reproducible builds** — for a privacy app this is trust infrastructure, not a nicety. Users should be able to verify the binary matches the published source.
- **First-run experience** — with no account, setup means generating a keypair. If that reads as confusing or scary, users are lost in the first 60 seconds.
- **Data export & migration** — take everything with you, in an open format
- **Mobile battery & data controls** — background behavior, cellular limits
- **Crash reporting** — opt-in only, no identifiers